

The background features a large white circle in the center, partially overlapping a light blue area on the left and a light pink area on the right. A large, dark blue shape is positioned at the bottom, also overlapping the white circle.

APPLIED DATA SCIENCE CAPSTONE

INTRODUCTION

This project integrates SQL queries, exploratory data analysis (EDA), and machine learning to analyze datasets. Data is stored and queried in SQL, explored with Python for insights, and modeled using Logistic Regression, SVM, Decision Trees, and KNN. Model performance is evaluated via accuracy and confusion matrices.

GitHub link :<https://github.com/VibeenaR/Coursera-Lab>



DATA COLLECTION

DATA IS COLLECTED FROM MULTIPLE SOURCES, INCLUDING CSV FILES AND ONLINE REPOSITORIES. RELEVANT DATASETS ARE IMPORTED INTO PYTHON AND STORED IN A SQLITE DATABASE FOR EFFICIENT QUERYING AND MANAGEMENT. SQL QUERIES ARE USED TO EXTRACT, FILTER, AND AGGREGATE THE DATA FOR ANALYSIS.



DATA WRANGLING

DATA IS CLEANED AND PREPARED FOR ANALYSIS. STEPS INCLUDE HANDLING MISSING VALUES, CONVERTING DATA TYPES, RENAMING COLUMNS, STANDARDIZING FEATURES, AND CREATING NEW DERIVED COLUMNS. TOOLS LIKE PANDAS AND NUMPY ARE USED TO TRANSFORM RAW DATA INTO A STRUCTURED FORMAT SUITABLE FOR EXPLORATORY DATA ANALYSIS AND MACHINE LEARNING.

DATA COLLECTION

Import Libraries and Define Auxiliary Functions

We will import the following libraries into the lab

```
In [1]: # Requests allows us to make HTTP requests which we will use to get data from an API
import requests
# Pandas is a software Library written for the Python programming Language for data manipulation and analysis.
import pandas as pd
# NumPy is a Library for the Python programming Language, adding support for Large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays
import numpy as np
# Datetime is a Library that allows us to represent dates
import datetime

# Setting this option will print all columns of a dataframe
pd.set_option('display.max_columns', None)
# Setting this option will print all of the data in a feature
pd.set_option('display.max_colwidth', None)
```

Below we will define a series of helper functions that will help us use the API to extract information using identification numbers in the launch data.

From the `rocket` column we would like to learn the booster name.

```
In [2]: # Takes the dataset and uses the rocket column to call the API and append the data to the List
def getBoosterVersion(data):
    for x in data['rocket']:
        if x:
            response = requests.get("https://api.spacexdata.com/v4/rockets/"+str(x)).json()
            BoosterVersion.append(response['name'])
```

From the `launchpad` we would like to know the name of the launch site being used, the longitude, and the latitude.

```
In [3]: # Takes the dataset and uses the launchpad column to call the API and append the data to the List
def getLaunchSite(data):
    for x in data['launchpad']:
        if x:
            response = requests.get("https://api.spacexdata.com/v4/launchpads/"+str(x)).json()
            Longitude.append(response['longitude'])
            Latitude.append(response['latitude'])
            LaunchSite.append(response['name'])
```

DATA WRANGLING

Import Libraries and Define Auxiliary Functions

We will import the following libraries.

```
In [2]: # Pandas is a software library written for the Python programming language for data manipulation and analysis.
import pandas as pd
#NumPy is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays
import numpy as np
```

Data Analysis

Load Space X dataset, from last section.

```
In [3]: df=pd.read_csv("https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_1.csv")
df.head(10)
```

```
Out[3]:
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Longitude	Latitude
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.577366	28.561857
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.577366	28.561857
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.577366	28.561857
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.610829	34.632093
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.577366	28.561857
5	6	2014-01-06	Falcon 9	3325.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1005	-80.577366	28.561857
6	7	2014-04-18	Falcon 9	2296.000000	ISS	CCAFS SLC 40	True Ocean	1	False	False	True	NaN	1.0	0	B1006	-80.577366	28.561857
7	8	2014-07-14	Falcon 9	1316.000000	LEO	CCAFS SLC 40	True Ocean	1	False	False	True	NaN	1.0	0	B1007	-80.577366	28.561857
8	9	2014-08-05	Falcon 9	4535.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1008	-80.577366	28.561857
9	10	2014-09-07	Falcon 9	4428.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1011	-80.577366	28.561857

Identify and calculate the percentage of the missing values in each attribute

EDA AND INTERACTIVE VISUAL ANALYTICS METHODOLOGY

In this project, exploratory data analysis (EDA) is performed to understand the dataset's structure, distributions, and relationships between features. Missing values, duplicates, and anomalies are identified and addressed. Interactive visual analytics using tools like Plotly and Folium enable dynamic exploration of patterns, trends, and spatial information, such as launch site locations and success/failure outcomes, supporting feature selection and insights for predictive modeling.

CONTD...

Load the dataframe

Load the data

```
In [4]: from js import fetch
import io

URL1 = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_2.csv"
resp1 = await fetch(URL1)
text1 = io.BytesIO((await resp1.arrayBuffer()).to_py())
data = pd.read_csv(text1)
```

```
In [5]: data.head()
```

```
Out[5]:
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Longitude	Latitude	Class
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.577366	28.561857	0
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.577366	28.561857	0
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.577366	28.561857	0
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.610829	34.632093	0
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.577366	28.561857	0

```
In [6]: URL2 = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_3.csv"
resp2 = await fetch(URL2)
text2 = io.BytesIO((await resp2.arrayBuffer()).to_py())
X = pd.read_csv(text2)
```

```
In [7]: X.head(100)
```

```
Out[7]:
```

	FlightNumber	PayloadMass	Flights	Block	ReusedCount	Orbit_ES-L1	Orbit_GEO	Orbit_GTO	Orbit_HEO	Orbit_ISS	...	Serial_B1058	Serial_B1059	Serial_B1060	Serial_B1062	GridFins_False	GridFins_True	Reused_False	Reused_True
0	1.0	6104.959412	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0
1	2.0	525.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0
2	3.0	677.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0

CONTD...

TASK 1

Create a NumPy array from the column `Class` in `data`, by applying the method `to_numpy()` then assign it to the variable `Y`, make sure the output is a Pandas series (only one bracket diff name of column!).

```
In [8]: # Extract the 'Class' column as a Pandas Series (single bracket)
        y_series = data['Class']

        # Convert the Pandas Series to a NumPy array
        Y = y_series.to_numpy()

        # Display the first few elements to check
        print(Y[:10])

[0 0 0 0 0 0 1 1 0 0]
```

TASK 2

Standardize the data in `X` then reassign it to the variable `X` using the transform provided below.

```
In [9]: # students get this
        transform = preprocessing.StandardScaler()
```

We split the data into training and testing data using the function `train_test_split`. The training data is divided into validation data, a second set used for training data; then the models are trained and hyperparameters are selected using the function `GridSearchCV`.

TASK 3

Use the function `train_test_split` to split the data `X` and `Y` into training and test data. Set the parameter `test_size` to 0.2 and `random_state` to 2. The training data and test data should be assigned to the following labels.

```
X_train, X_test, Y_train, Y_test

In [11]: from sklearn.model_selection import train_test_split

        # Split X and Y into training and test sets
        X_train, X_test, Y_train, Y_test = train_test_split(
            X, Y, test_size=0.2, random_state=2
        )
```

CONTD...

Out[12]: (18,)

TASK 4

Create a logistic regression object then create a GridSearchCV object `logreg_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
In [13]: parameters = {'C':[0.01,0.1,1],
                      'penalty':['l2'],
                      'solver':['lbfgs']}
```

```
In [14]: parameters = {"C":[0.01,0.1,1],'penalty':['l2'], 'solver':['lbfgs']}# L1 Lasso L2 ridge
lr=LogisticRegression()
```

We output the `GridSearchCV` object for logistic regression. We display the best parameters using the data attribute `best_params_` and the accuracy on the validation data using the data attribute `best_score_`.

```
In [16]: # Define the Logistic regression model
logreg = LogisticRegression(max_iter=1000) # increase max_iter to ensure convergence

# Define hyperparameter grid
param_grid = {
    'C': [0.01, 0.1, 1, 10, 100], # regularization strength
    'solver': ['liblinear', 'lbfgs'] # solvers compatible with LogisticRegression
}

# Create GridSearchCV object
grid_search = GridSearchCV(estimator=logreg, param_grid=param_grid, cv=5)

# Fit GridSearchCV on training data
grid_search.fit(X_train, Y_train)

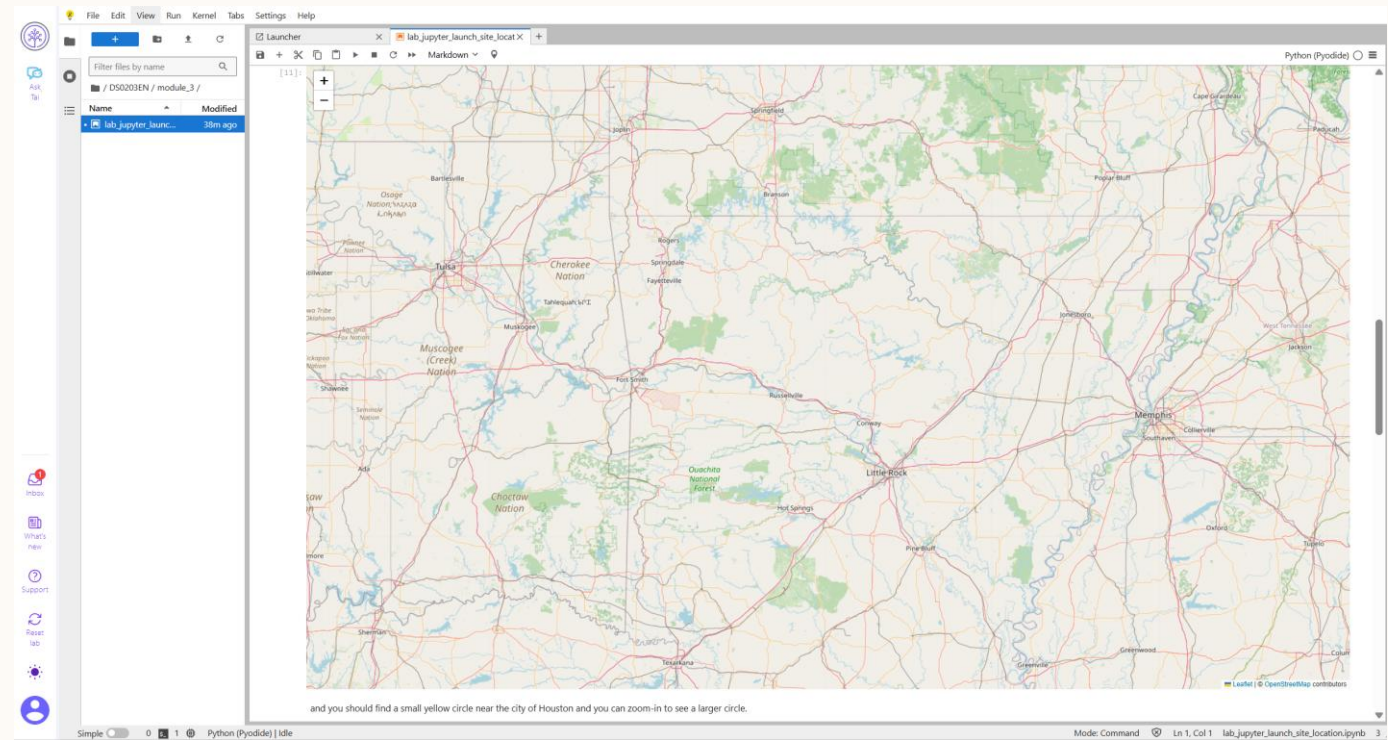
# Output the GridSearchCV object
print(grid_search)

# Display the best hyperparameters
print("Best Parameters:", grid_search.best_params_)

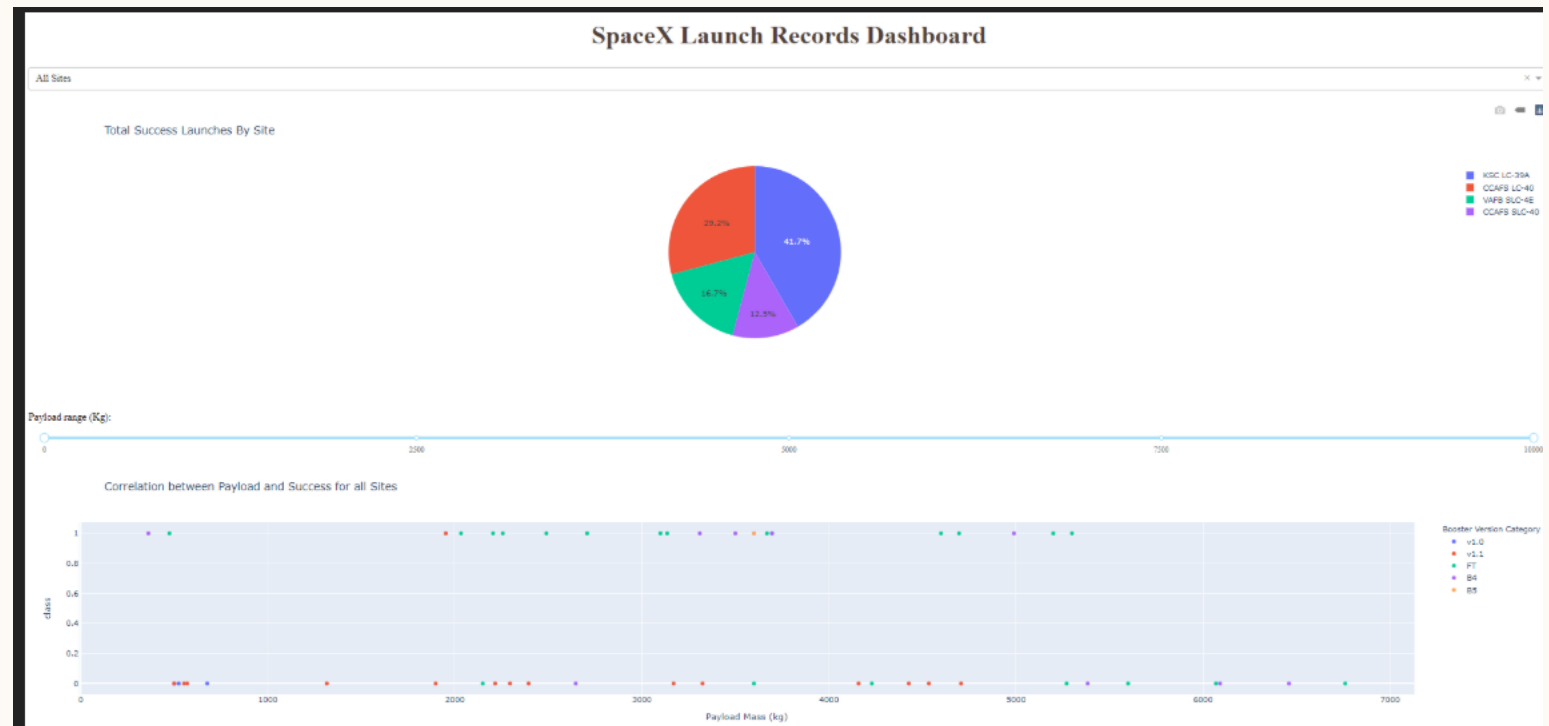
# Display the best validation score (mean accuracy across folds)
print("Best Validation Accuracy:", grid_search.best_score_)
```

```
GridSearchCV(cv=5, estimator=LogisticRegression(max_iter=1000),
             param_grid={'C': [0.01, 0.1, 1, 10, 100],
                          'solver': ['liblinear', 'lbfgs']})
Best Parameters: {'C': 0.1, 'solver': 'lbfgs'}
Best Validation Accuracy: 0.8342857142857143
```

CONTD...



CONTD...



PREDICTIVE ANALYSIS METHODOLOGY SLIDES

Predictive analysis involves selecting relevant features from the dataset, preprocessing the data by handling missing values and standardizing features, and applying supervised learning models such as Logistic Regression, SVM, Decision Tree, and KNN. Models are tuned using GridSearchCV, trained on the training set, evaluated on the test set, and compared to select the best-performing model for final predictions.



Load the data

```
In [5]: data.head()
```

```
In [7]: x.head(100)
```

[illegible]

CONTD...

TASK 1

Create a NumPy array from the column `Class` in `data`, by applying the method `to_numpy()` then assign it to the variable `Y`, make sure the output is a Pandas series (only one bracket diff name of column!).

```
In [8]: # Extract the 'Class' column as a Pandas Series (single bracket)
        y_series = data['Class']

        # Convert the Pandas Series to a NumPy array
        Y = y_series.to_numpy()

        # Display the first few elements to check
        print(Y[:10])

[0 0 0 0 0 1 1 0 0]
```

TASK 2

Standardize the data in `X` then reassign it to the variable `X` using the transform provided below.

```
In [9]: # students get this
        transform = preprocessing.StandardScaler()
```

We split the data into training and testing data using the function `train_test_split`. The training data is divided into validation data, a second set used for training data; then the models are trained and hyperparameters are selected using the function `GridSearchCV`.

TASK 3

Use the function `train_test_split` to split the data `X` and `Y` into training and test data. Set the parameter `test_size` to 0.2 and `random_state` to 2. The training data and test data should be assigned to the following labels.

`X_train, X_test, Y_train, Y_test`

```
In [11]: from sklearn.model_selection import train_test_split

        # Split X and Y into training and test sets
        X_train, X_test, Y_train, Y_test = train_test_split(
            X, Y, test_size=0.2, random_state=2
        )

        # Display the shapes to confirm
        print("X_train shape:", X_train.shape)
        print("X_test shape:", X_test.shape)
        print("Y_train shape:", Y_train.shape)
        print("Y_test shape:", Y_test.shape)
```

```
X_train shape: (72, 83)
X_test shape: (18, 83)
Y_train shape: (72,)
Y_test shape: (18,)
```

we can see we only have 18 test samples.

```
In [12]: Y_test.shape
```

```
Out[12]: (18,)
```

TASK 4

CONTD...

we can see we only have 18 test samples.

```
In [12]: Y_test.shape
```

```
Out[12]: (18,)
```

TASK 4

Create a logistic regression object then create a GridSearchCV object `logreg_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
In [13]: parameters = {'C': [0.01, 0.1, 1],
                      'penalty': ['l2'],
                      'solver': ['lbfgs']}
```

```
In [14]: parameters = {'C': [0.01, 0.1, 1], 'penalty': ['l2'], 'solver': ['lbfgs']} # L1 lasso L2 ridge
lr = LogisticRegression()
```

We output the `GridSearchCV` object for logistic regression. We display the best parameters using the data attribute `best_params_` and the accuracy on the validation data using the data attribute `best_score_`.

```
In [16]: # Define the Logistic regression model
logreg = LogisticRegression(max_iter=1000) # Increase max_iter to ensure convergence

# Define hyperparameter grid
param_grid = {
    'C': [0.01, 0.1, 1, 10, 100], # regularization strength
    'solver': ['liblinear', 'lbfgs'] # solvers compatible with LogisticRegression
}

# Create GridSearchCV object
grid_search = GridSearchCV(estimator=logreg, param_grid=param_grid, cv=5)

# Fit GridSearchCV on training data
grid_search.fit(X_train, Y_train)

# Output the GridSearchCV object
print(grid_search)

# Display the best hyperparameters
print("Best Parameters:", grid_search.best_params_)

# Display the best validation score (mean accuracy across folds)
print("Best Validation Accuracy:", grid_search.best_score_)

GridSearchCV(cv=5, estimator=LogisticRegression(max_iter=1000),
             param_grid={'C': [0.01, 0.1, 1, 10, 100],
                          'solver': ['liblinear', 'lbfgs']})
Best Parameters: {'C': 0.1, 'solver': 'lbfgs'}
Best Validation Accuracy: 0.8342857142857143
```

TASK 5

Calculate the accuracy on the test data using the method `score`:

```
In [17]: # Get the best estimator from GridSearchCV
best_logreg = grid_search.best_estimator_

# Calculate accuracy on the test set
test_accuracy = best_logreg.score(X_test, Y_test)
```

CONTD...

TASK 5

Calculate the accuracy on the test data using the method `score`:

```
In [17]: # Get the best estimator from GridSearchCV
best_logreg = grid_search.best_estimator_

# Calculate accuracy on the test set
test_accuracy = best_logreg.score(X_test, Y_test)

print("Test Set Accuracy:", test_accuracy)
```

Test Set Accuracy: 0.8333333333333334

Lets look at the confusion matrix

```
In [19]: from sklearn.metrics import confusion_matrix

# Get predictions on the test set
Y_pred = best_logreg.predict(X_test)

# Compute the confusion matrix
cm = confusion_matrix(Y_test, Y_pred)

print("Confusion Matrix:")
print(cm)
```

Confusion Matrix:
[[3 3]
[0 12]]

Examining the confusion matrix, we see that logistic regression can distinguish between the different classes. We see that the problem is false positives.

Overview:

True Postive - 12 (True label is landed, Predicted label is also landed)

False Postive - 3 (True label is not landed, Predicted label is landed)

TASK 6

Create a support vector machine object then create a `GridSearchCV` object `svm_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
In [22]: from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

# Create the SVM object
svm = SVC()

# Define the hyperparameter grid
parameters = {
    'C': [0.1, 1, 10, 100],          # Regularization parameter
    'kernel': ['linear', 'rbf']      # Kernel function
```

CONTD...

False Positive - 2 (True label is not landed, Predicted label is landed)

TASK 6

Create a support vector machine object then create a `GridSearchCV` object `svm_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
In [22]: from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

# Create the SVM object
svm = SVC()

# Define the hyperparameter grid
parameters = {
    'C': [0.1, 1, 10, 100],      # Regularization parameter
    'kernel': ['linear', 'rbf'],  # Kernel types
    'gamma': ['scale', 'auto']   # Kernel coefficient for rbf
}

# Create GridSearchCV object with cv=10
svm_cv = GridSearchCV(estimator=svm, param_grid=parameters, cv=10)

# Fit the GridSearchCV object to the training data
svm_cv.fit(X_train, Y_train)

# Display the best parameters
print("Best Parameters:", svm_cv.best_params_)

# Display the best cross-validation score
print("Best CV Accuracy:", svm_cv.best_score_)

Best Parameters: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
Best CV Accuracy: 0.8196428571428571
```

TASK 7

Calculate the accuracy on the test data using the method `score`:

```
In [23]: # Get the best SVM estimator from GridSearchCV
best_svm = svm_cv.best_estimator_

# Calculate accuracy on the test set
test_accuracy = best_svm.score(X_test, Y_test)

print("SVM Test Set Accuracy:", test_accuracy)
```

SVM Test Set Accuracy: 0.7777777777777778

We can plot the confusion matrix

```
In [24]: from sklearn.metrics import confusion_matrix

# Predict labels for the test set using the best SVM model
Y_pred_svm = best_svm.predict(X_test)

# Compute the confusion matrix
cm_svm = confusion_matrix(Y_test, Y_pred_svm)

print("SVM Confusion Matrix:")
print(cm_svm)
```

CONTD...

```
Best Parameters: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}  
Best CV Accuracy: 0.8196428571428571
```

TASK 7

Calculate the accuracy on the test data using the method `score`:

```
In [23]: # Get the best SVM estimator from GridSearchCV  
best_svm = svm_cv.best_estimator_  
  
# Calculate accuracy on the test set  
test_accuracy = best_svm.score(X_test, Y_test)  
  
print("SVM Test Set Accuracy:", test_accuracy)
```

```
SVM Test Set Accuracy: 0.7777777777777778
```

We can plot the confusion matrix

```
In [24]: from sklearn.metrics import confusion_matrix  
  
# Predict labels for the test set using the best SVM model  
Y_pred_svm = best_svm.predict(X_test)  
  
# Compute the confusion matrix  
cm_svm = confusion_matrix(Y_test, Y_pred_svm)  
  
print("SVM Confusion Matrix:")  
print(cm_svm)
```

```
SVM Confusion Matrix:  
[[ 3  3]  
 [ 1 11]]
```

TASK 8

Create a decision tree classifier object then create a `GridSearchCV` object `tree_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
In [26]: from sklearn.tree import DecisionTreeClassifier  
from sklearn.model_selection import GridSearchCV  
  
# Create a Decision Tree classifier object  
tree = DecisionTreeClassifier(random_state=2)
```

CONTD...

```
print("SVM Confusion Matrix:")
print(cm_svm)
```

```
SVM Confusion Matrix:
[[ 3  3]
 [ 1 11]]
```

TASK 8

Create a decision tree classifier object then create a `GridSearchCV` object `tree_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
In [26]: from sklearn.tree import DecisionTreeClassifier
         from sklearn.model_selection import GridSearchCV

         # Create a Decision Tree classifier object
         tree = DecisionTreeClassifier(random_state=2)

         # Define the hyperparameter grid
         parameters = {
             'criterion': ['gini', 'entropy'], # function to measure quality of split
             'max_depth': [None, 2, 4, 6, 8, 10], # maximum depth of the tree
             'min_samples_split': [2, 5, 10], # minimum samples required to split a node
             'min_samples_leaf': [1, 2, 4] # minimum samples required at a leaf node
         }

         # Create GridSearchCV object with 10-fold cross-validation
         tree_cv = GridSearchCV(estimator=tree, param_grid=parameters, cv=10)

         # Fit the GridSearchCV object to the training data
         tree_cv.fit(X_train, Y_train)

         # Display the best parameters
         print("Best Parameters:", tree_cv.best_params_)

         # Display the best cross-validation score
         print("Best CV Accuracy:", tree_cv.best_score_)

         Best Parameters: {'criterion': 'gini', 'max_depth': None, 'min_samples_leaf': 4, 'min_samples_split': 2}
         Best CV Accuracy: 0.8714285714285713
```

TASK 9

Calculate the accuracy of `tree_cv` on the test data using the method `score`:

CONTD...

TASK 9

Calculate the accuracy of `tree_cv` on the test data using the method `score`:

```
In [27]: # Get the best estimator from GridSearchCV
best_tree = tree_cv.best_estimator_

# Calculate accuracy on the test set
test_accuracy = best_tree.score(X_test, Y_test)

print("Decision Tree Test Set Accuracy:", test_accuracy)
```

Decision Tree Test Set Accuracy: 0.7777777777777778

We can plot the confusion matrix

```
In [28]: from sklearn.metrics import confusion_matrix

# Predict labels on the test set using the best Decision Tree
Y_pred_tree = best_tree.predict(X_test)

# Compute the confusion matrix
cm_tree = confusion_matrix(Y_test, Y_pred_tree)

print("Decision Tree Confusion Matrix:")
print(cm_tree)
```

Decision Tree Confusion Matrix:

```
[[ 3  3]
 [ 1 11]]
```

TASK 10

Create a k nearest neighbors object then create a `GridSearchCV` object `knn_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
In [29]: from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV

# Create a KNN classifier object
knn = KNeighborsClassifier()

# Define the hyperparameter grid
parameters = {
```

CONTD...

Decision Tree Confusion Matrix:
[[3 3]
 [1 11]]

TASK 10

Create a k nearest neighbors object then create a `GridSearchCV` object `knn_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
In [29]: from sklearn.neighbors import KNeighborsClassifier
         from sklearn.model_selection import GridSearchCV

         # Create a KNN classifier object
         knn = KNeighborsClassifier()

         # Define the hyperparameter grid
         parameters = {
             'n_neighbors': [3, 5, 7, 9, 11], # number of neighbors
             'weights': ['uniform', 'distance'], # uniform or distance-based weighting
             'metric': ['euclidean', 'manhattan'] # distance metric
         }

         # Create GridSearchCV object with 10-fold cross-validation
         knn_cv = GridSearchCV(estimator=knn, param_grid=parameters, cv=10)

         # Fit the GridSearchCV object to the training data
         knn_cv.fit(X_train, Y_train)

         # Display the best parameters
         print("Best Parameters:", knn_cv.best_params_)

         # Display the best cross-validation score
         print("Best CV Accuracy:", knn_cv.best_score_)
```

```
Best Parameters: {'metric': 'manhattan', 'n_neighbors': 11, 'weights': 'uniform'}
Best CV Accuracy: 0.8482142857142858
```

TASK 11

Calculate the accuracy of `knn_cv` on the test data using the method `score`:

```
In [ ]:
```

We can plot the confusion matrix

EDA WITH SQL

Exploratory Data Analysis using SQL involves querying datasets to filter, aggregate, and summarize data. SQL helps uncover patterns, relationships, and trends efficiently, enabling analysis of categorical and numerical variables, identifying missing values, and preparing the dataset for further visualization or predictive modeling.



QUERIES USED

1. %%sql
SELECT DISTINCT "Launch Site"
FROM SPACEXTABLE;

2. %%sql
SELECT *
FROM SPACEXTABLE
WHERE "Launch Site" LIKE 'CCA%'
LIMIT 5;



QUERIES USED

3. %%%sql

```
SELECT SUM(CAST("Payload Mass (kg)" AS  
FLOAT)) AS Total_Payload_Mass  
FROM SPACEXTABLE  
WHERE "Customer" LIKE '%NASA (CRS)%';
```

4. %%%sql

```
SELECT AVG(CAST("Payload Mass (kg)" AS  
FLOAT)) AS Avg_Payload_Mass  
FROM SPACEXTABLE  
WHERE "Booster Version" = 'F9 v1.1';
```



QUERIES USED

5. %%%sql

```
SELECT MIN("Date") AS  
First_Successful_Landing_Date  
FROM SPACEXTABLE  
WHERE "Landing Outcome" = 'Success (ground pad)';
```

6. %%%sql

```
SELECT MIN("Date") AS  
First_Successful_Landing_Date  
FROM SPACEXTABLE  
WHERE LOWER("Landing Outcome") = 'success  
(ground pad)';
```



QUERIES USED

7. %%%sql

```
SELECT MIN("Date") AS  
First_Successful_Landing_Date  
FROM SPACEXTABLE  
WHERE "Landing Outcome" LIKE '%ground pad%';
```

8. %%%sql

```
SELECT DISTINCT "Booster Version"  
FROM SPACEXTABLE  
WHERE "Landing Outcome" = 'Success (drone ship)'  
AND CAST("Payload Mass (kg)" AS FLOAT) > 4000  
AND CAST("Payload Mass (kg)" AS FLOAT) < 6000;
```



QUERIES USED

9. %%%sql

```
SELECT "Mission Outcome", COUNT(*) AS Total  
FROM SPACEXTABLE  
WHERE "Mission Outcome" IS NOT NULL  
GROUP BY "Mission Outcome";
```

10. %%%sql

```
SELECT "Booster Version"  
FROM SPACEXTABLE  
WHERE CAST("Payload Mass (kg)" AS FLOAT) = (  
    SELECT MAX(CAST("Payload Mass (kg)" AS FLOAT))  
    FROM SPACEXTABLE  
);
```



QUERIES USED

11. %%sql

```
SELECT
    substr("Date", 6, 2) AS Month,
    "Landing Outcome",
    "Booster Version",
    "Launch Site"
FROM SPACEXTABLE
WHERE "Landing Outcome" LIKE '%Failure%'
    AND "Landing Outcome" LIKE '%drone ship%'
    AND substr("Date", 1, 4) = '2015';
```

12. %%sql

```
SELECT "Landing Outcome", COUNT(*) AS Outcome_Count
FROM SPACEXTABLE
WHERE "Date" BETWEEN '2010-06-04' AND '2017-03-20'
    AND "Landing Outcome" IS NOT NULL
GROUP BY "Landing Outcome"
ORDER BY Outcome_Count DESC;
```



CONTD...

Task 1

Display the names of the unique launch sites in the space mission

```
In [10]: %%sql
SELECT DISTINCT "Launch Site"
FROM SPACEXTABLE;

* sqlite:///my_data1.db
Done.
Out[10]: "Launch Site"
Launch Site
```

Task 2

Display 5 records where launch sites begin with the string 'CCA'

```
In [11]: %%sql
SELECT *
FROM SPACEXTABLE
WHERE "Launch Site" LIKE 'CCA%'
LIMIT 5;

* sqlite:///my_data1.db
Done.
Out[11]: Date Time (UTC) Booster_Version Launch_Site Payload PAYLOAD_MASS_KG Orbit Customer Mission_Outcome Landing_Outcome
```

Task 3

Display the total payload mass carried by boosters launched by NASA (CRS)

```
In [13]: %%sql
SELECT SUM(CAST("Payload Mass (kg)" AS FLOAT)) AS Total_Payload_Mass
FROM SPACEXTABLE
WHERE "Customer" LIKE '%NASA (CRS)%';

* sqlite:///my_data1.db
Done.
Out[13]: Total_Payload_Mass
```

CONTD...

Out[13]: Total_Payload_Mass
0.0

Task 4

Display average payload mass carried by booster version F9 v1.1

In [14]:

```
%sql
SELECT AVG(CAST("Payload Mass (kg)" AS FLOAT)) AS Avg_Payload_Mass
FROM SPACEXTABLE
WHERE "Booster Version" = 'F9 v1.1';
```

* sqlite:///my_data1.db
Done.

Out[14]: Avg_Payload_Mass
None

Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint: Use min function

In [15]:

```
%sql
SELECT MIN("Date") AS First_Successful_Landing_Date
FROM SPACEXTABLE
WHERE "Landing Outcome" = 'Success (ground pad)';
```

* sqlite:///my_data1.db
Done.

Out[15]: First_Successful_Landing_Date
None

In [16]:

```
%sql
SELECT MIN("Date") AS First_Successful_Landing_Date
FROM SPACEXTABLE
WHERE LOWER("Landing Outcome") = 'success (ground pad)';
```

* sqlite:///my_data1.db
Done.

Out[16]: First_Successful_Landing_Date
None

CONTD...

```
* sqlite:///my_data1.db
Done.
Out[16]: First_Successful_Landing_Date
```

None

```
In [17]: %%sql
SELECT MIN("Date") AS First_Successful_Landing_Date
FROM SPACEXTABLE
WHERE "Landing Outcome" LIKE '%ground pad%';
```

```
* sqlite:///my_data1.db
Done.
Out[17]: First_Successful_Landing_Date
```

None

Task 6

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
In [18]: %%sql
SELECT DISTINCT "Booster Version"
FROM SPACEXTABLE
WHERE "Landing Outcome" = 'Success (drone ship)'
AND CAST("Payload Mass (kg)" AS FLOAT) > 4000
AND CAST("Payload Mass (kg)" AS FLOAT) < 6000;
```

```
* sqlite:///my_data1.db
Done.
Out[18]: "Booster Version"
```

Task 7

List the total number of successful and failure mission outcomes

```
In [19]: %%sql
SELECT "Mission Outcome", COUNT(*) AS Total
FROM SPACEXTABLE
WHERE "Mission Outcome" IS NOT NULL
GROUP BY "Mission Outcome";
```

```
* sqlite:///my_data1.db
Done.
Out[19]: "Mission Outcome" Total
```

Mission Outcome	Total
	101

CONTD...

[illegible]

CONTD...

Task 9

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

In [21]:

```
%%sql
SELECT
    substr("Date", 6, 2) AS Month,
    "Landing Outcome",
    "Booster Version",
    "Launch Site"
FROM SPACEXTABLE
WHERE "Landing Outcome" LIKE '%Failure%'
AND "Landing Outcome" LIKE '%drone ship%'
AND substr("Date", 1, 4) = '2015';
```

* sqlite:///my_data1.db
Done.

Out[21]:

Month	Landing Outcome	Booster Version	Launch Site
-------	-----------------	-----------------	-------------

Task 10

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

In [22]:

```
%%sql
SELECT "Landing Outcome", COUNT(*) AS Outcome_Count
FROM SPACEXTABLE
WHERE "Date" BETWEEN '2010-06-04' AND '2017-03-20'
AND "Landing Outcome" IS NOT NULL
GROUP BY "Landing Outcome"
ORDER BY Outcome_Count DESC;
```

* sqlite:///my_data1.db
Done.

Out[22]:

Landing Outcome	Outcome_Count
Landing Outcome	31

Reference Links

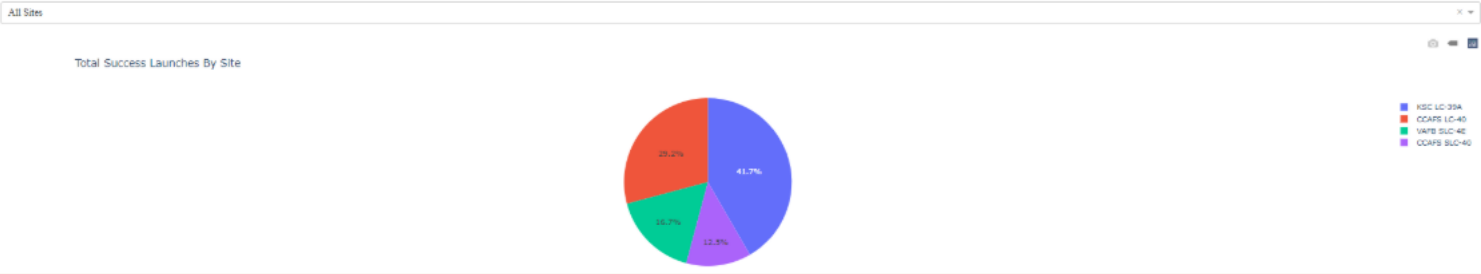
- [Hands-on Lab: String Patterns, Sorting and Grouping](#)
- [Hands-on Lab: Built-in functions](#)
- [Hands-on Lab: Sub-queries and Nested SELECT Statements](#)

EDA WITH VISUALIZATION

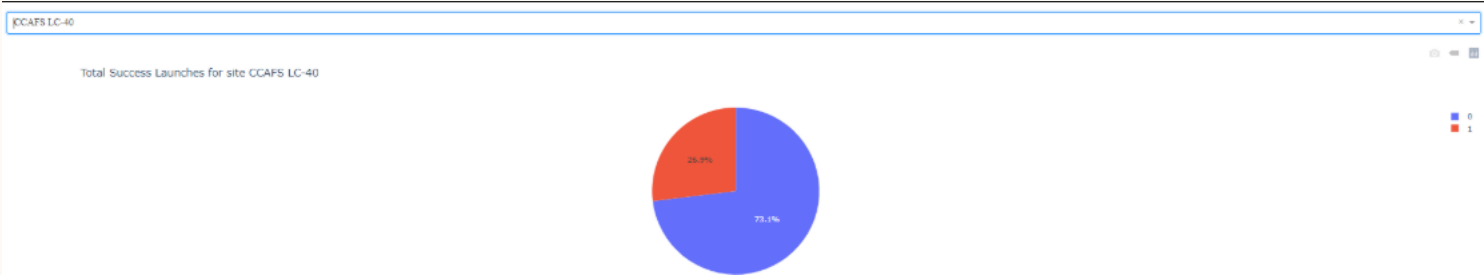
EDA with visualization uses tools like Matplotlib, Seaborn, Plotly, and Folium to explore data distributions, correlations, trends, and spatial information. Visualizations make insights intuitive, highlight anomalies or patterns, and support feature selection, aiding in the preparation of clean and structured data for machine learning models.



PIE CHART FOR ALL STATES



PIE CHART SELECTED STATES



CONTD...

The screenshot displays a JupyterLab environment with a file explorer on the left, a code editor in the center, and a map visualization at the bottom.

File Explorer: Shows a file named `lab_jupyter_launch...` modified 51m ago.

Code Editor: Contains the following Python code:

```
folium.Circle(coordinate, radius=1000, color='000000', fill=True).add_child(folium.Popup(...))
```

An example of folium.Marker:

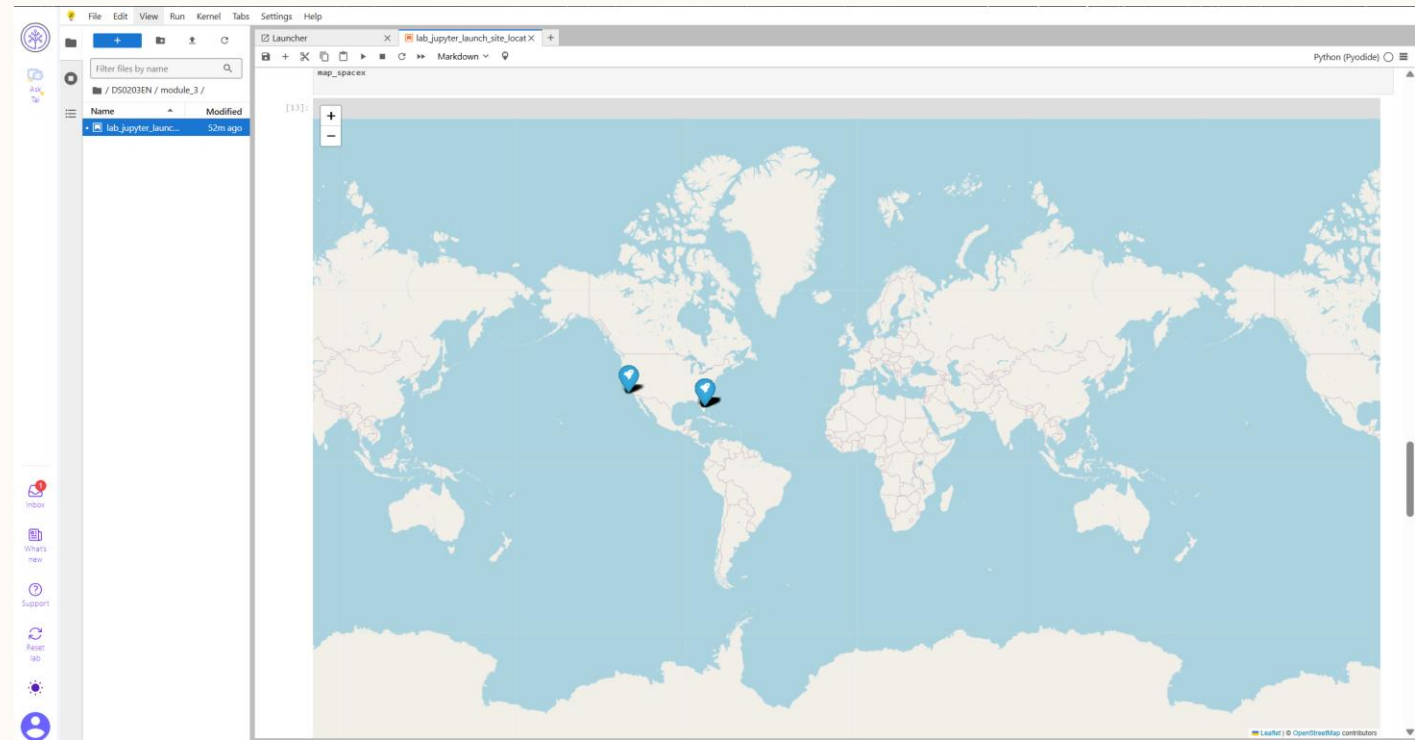
```
folium.Marker(coordinate, icon=DivIcon(icon_size=(20,20),icon_anchor=(0,0), html='<div style="font-size: 12; color:#d35400;"><b><div>' % 'label', ))
```

[12]: # Initial the map
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)
For each launch site, add a Circle object based on its coordinate (lat, long) values. In addition, add launch site name as a popup label

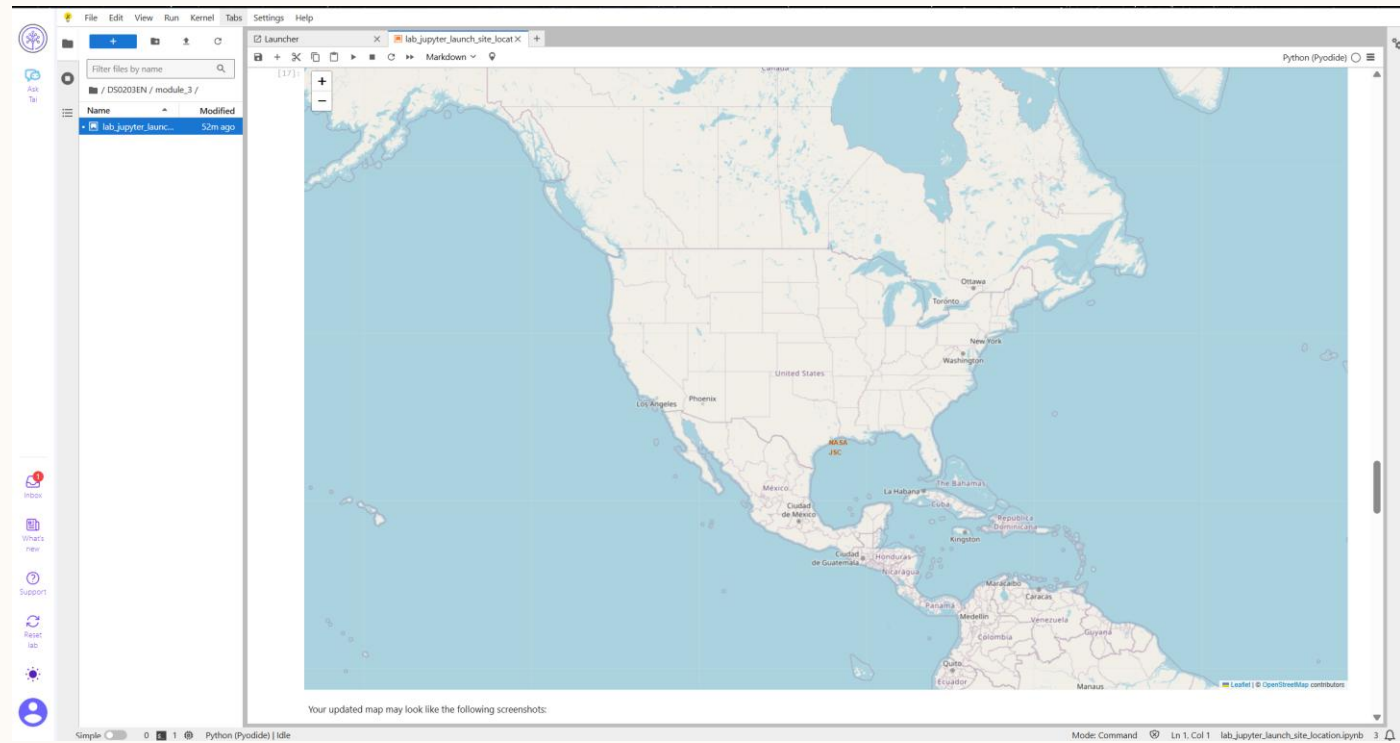
The generated map with marked launch sites should look similar to the following:

Now, you can explore the map by zoom-in/out the marked areas, and try to answer the following questions:

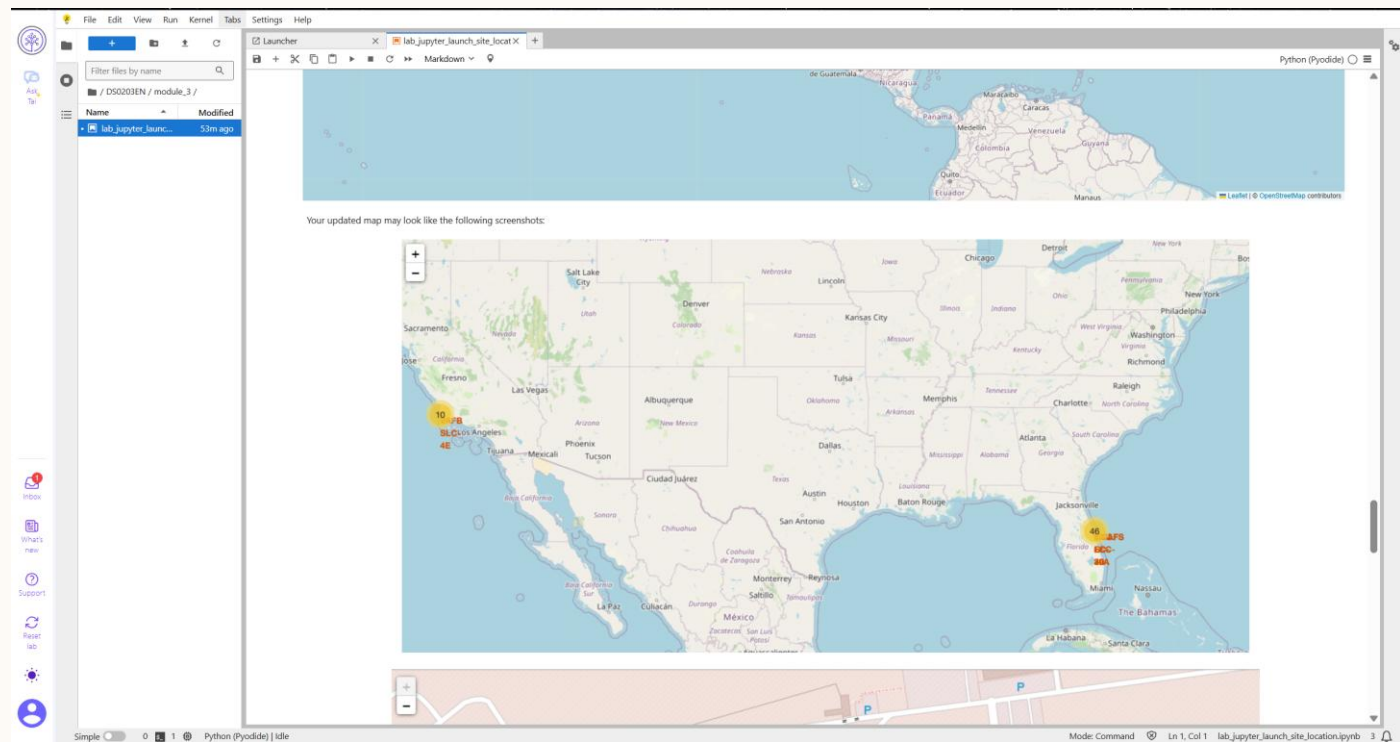
CONTD...



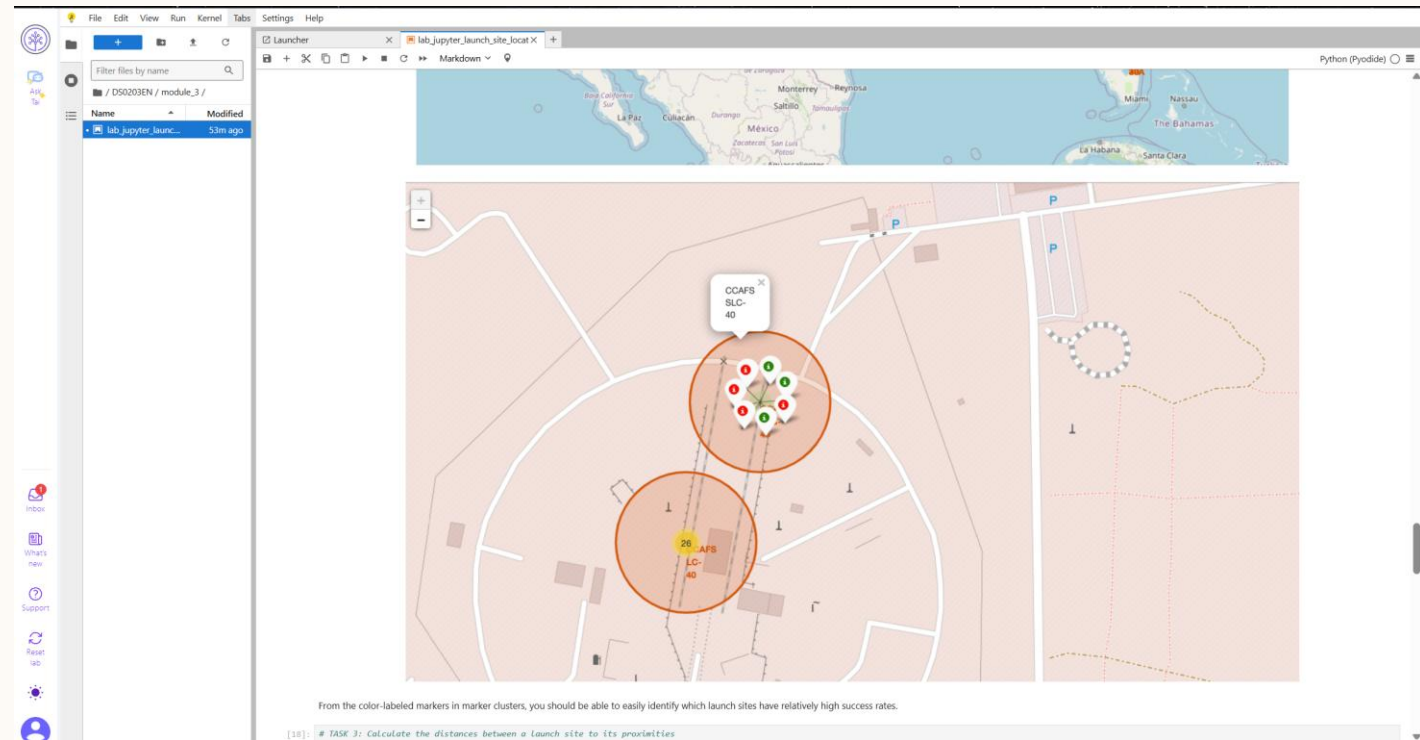
CONTD...



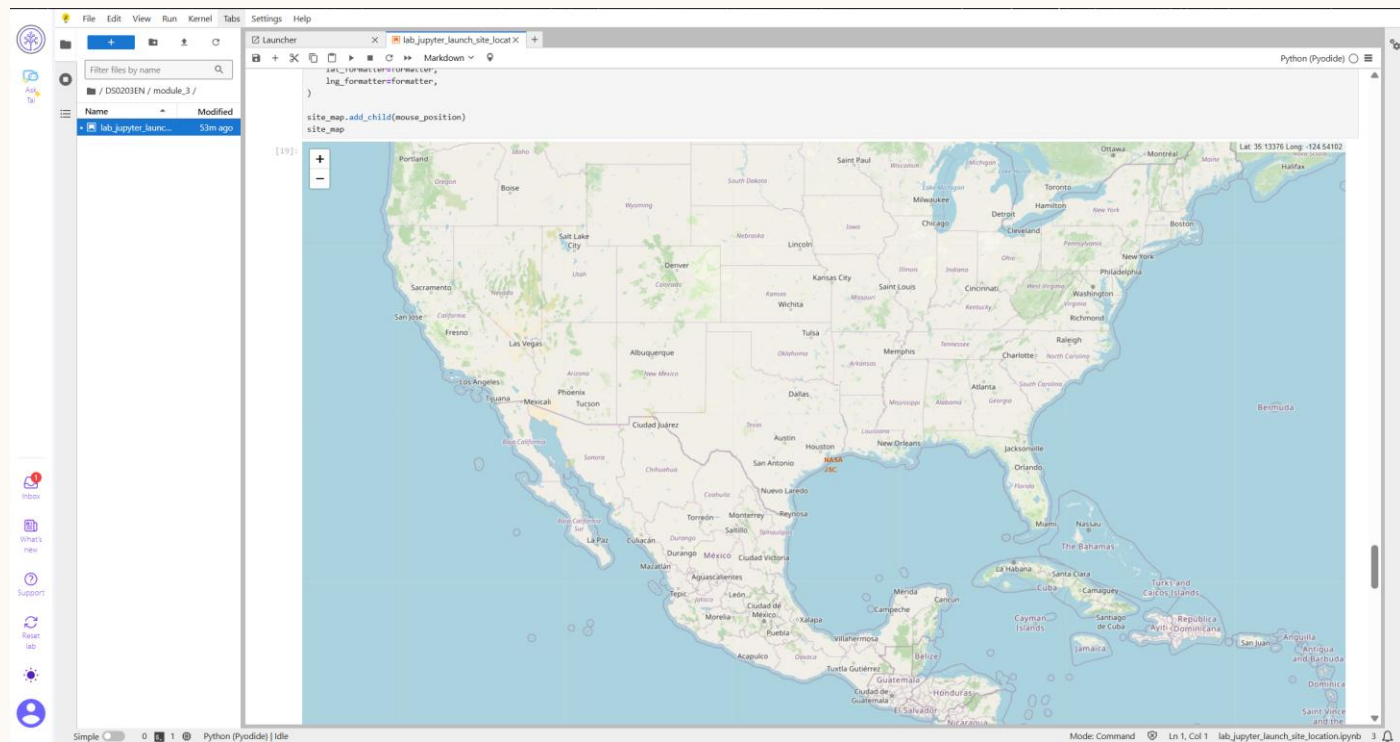
CONTD...



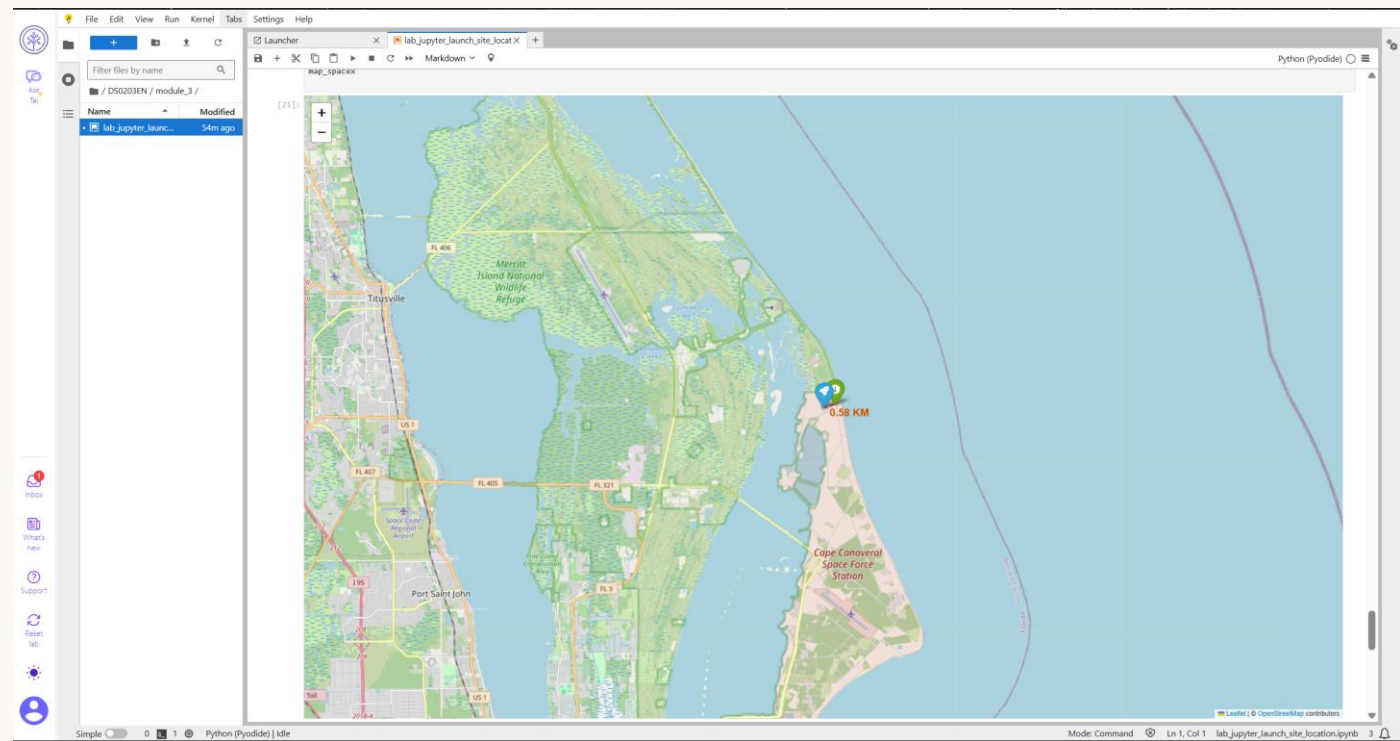
CONTD...



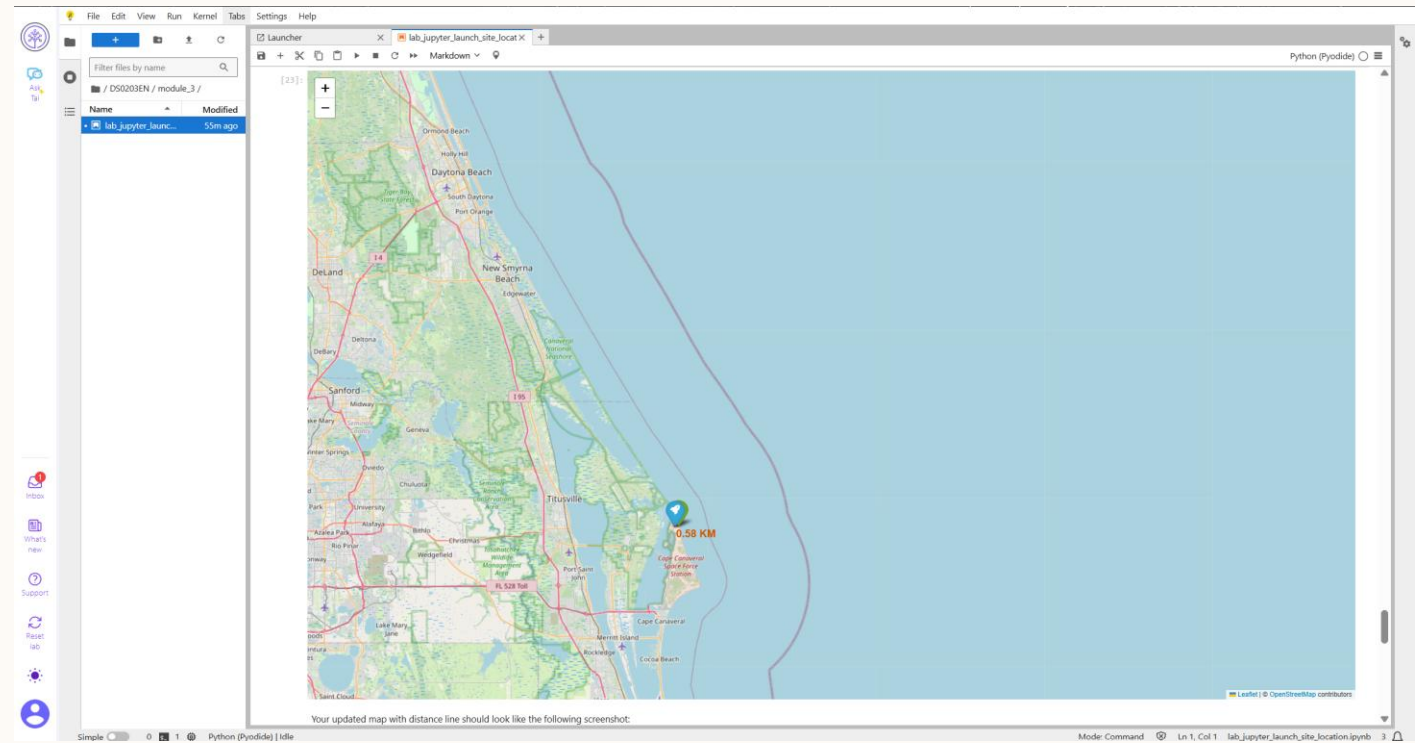
CONTD...



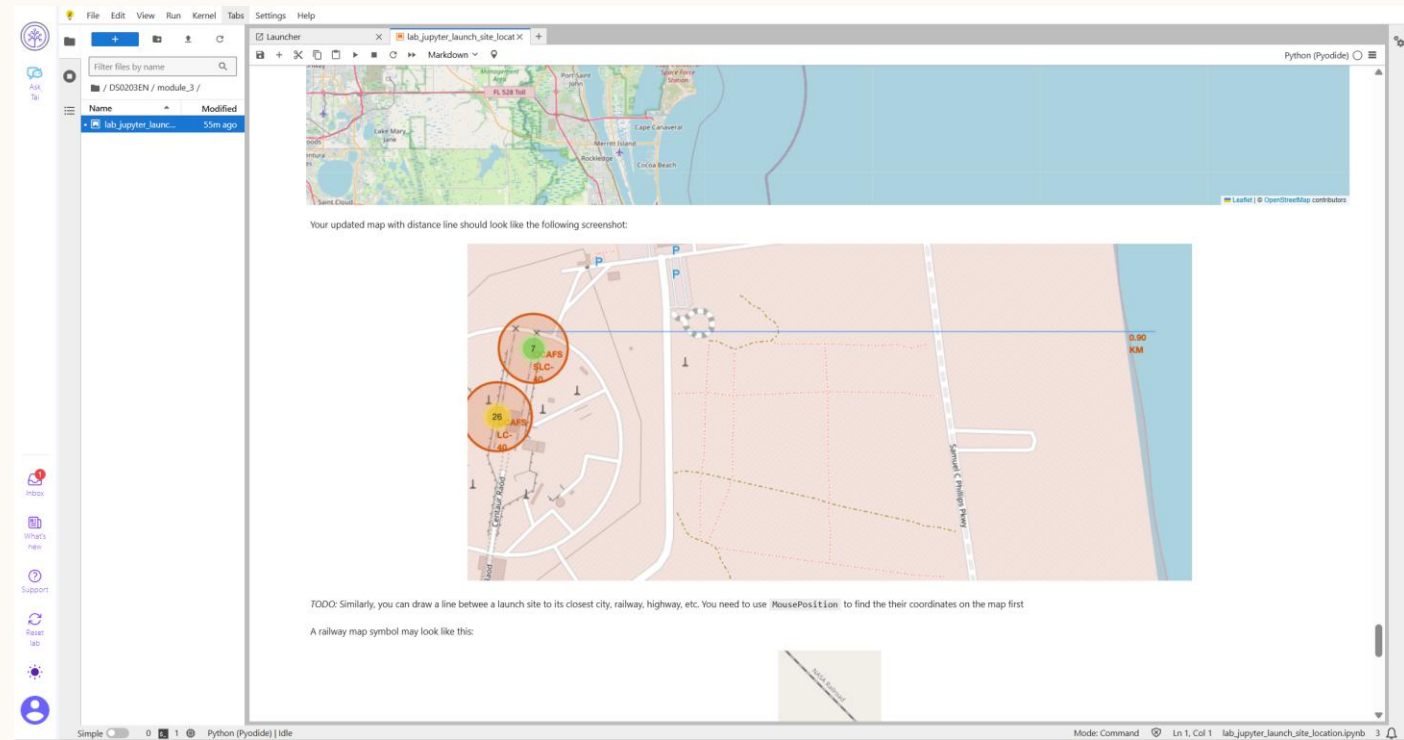
CONTD...



CONTD...



CONTD...



The screenshot shows a JupyterLab interface with a file browser on the left and a main workspace. The file browser shows a directory structure with a file named 'lab_jupyter_launch_site_location.ipynb'. The main workspace displays a map of a coastal area with a distance line drawn between two points. The distance is labeled '0.00 KM'. Below the map, there is a text box with the following content:

Your updated map with distance line should look like the following screenshot:



TODO: Similarly, you can draw a line between a launch site to its closest city, railway, highway, etc. You need to use `MousePosition` to find the their coordinates on the map first

A railway map symbol may look like this:



The bottom status bar indicates the mode is 'Command', the file is 'lab_jupyter_launch_site_location.ipynb', and the kernel is 'Python (Pyodide) | Idle'.

CONTD...

The screenshot shows a JupyterLab environment. On the left is a file explorer with a sidebar containing icons for Home, Info, What's New, Support, Reset Lab, and a user profile. The main area is divided into three panes. The top pane is a map viewer showing a satellite view of a coastal area with a red line drawn across it. The middle pane is a code editor with the following text:

TODO: Similarly, you can draw a line between a launch site to its closest city, railway, highway, etc. You need to use `MousePosition` to find the their coordinates on the map first

A railway map symbol may look like this:



A highway map symbol may look like this:



A city map symbol may look like this:



[24]: # Create a marker with distance to a closest city, railway, highway, etc.
Draw a line between the marker to the launch site

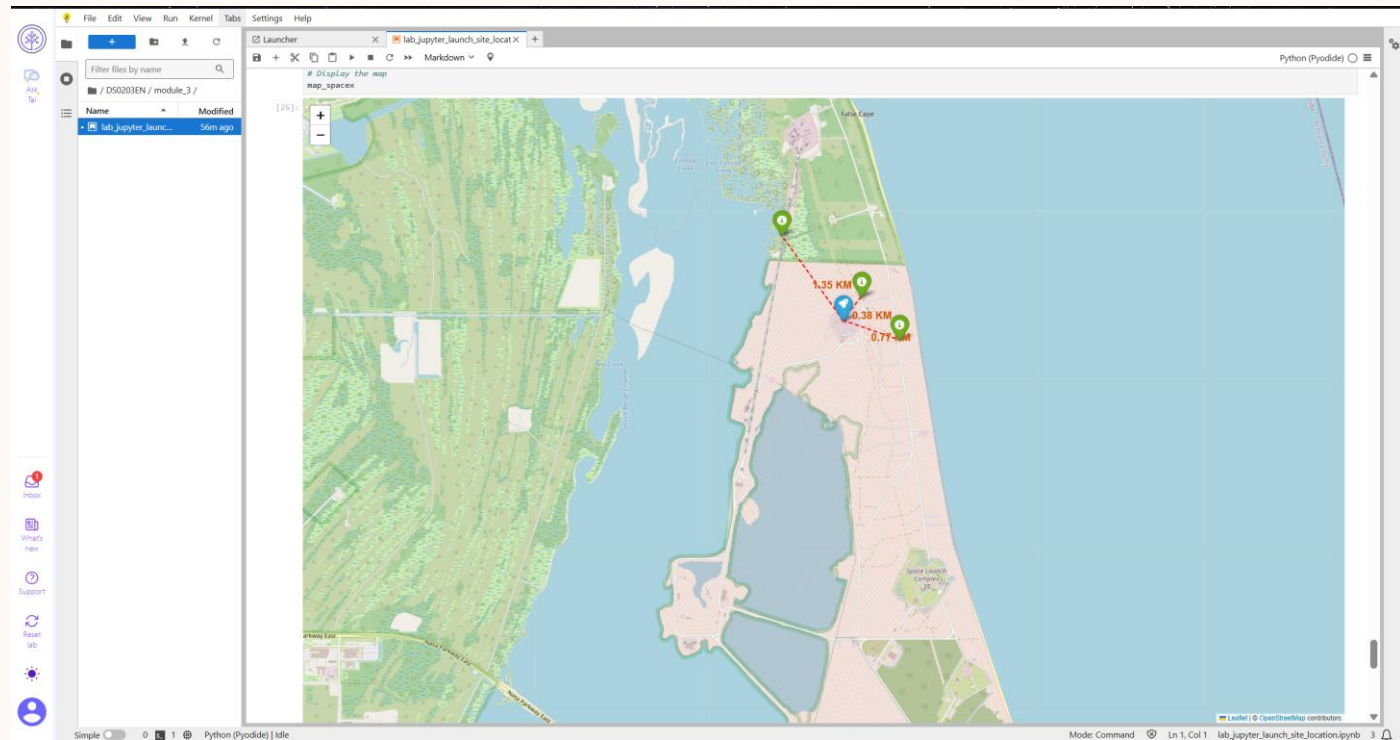
[25]: # Haversine function to calculate distance in km

```
def haversine(lat1, lon1, lat2, lon2):
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
    c = 2 * np.arcsin(np.sqrt(a))
    r = 6371
    return c * r
```

[26]: # Example launch site coordinates
launch_site = [28.5623, -80.5774] # Launch site

The bottom status bar shows 'Simple' mode, '0' lines, '1' column, 'Python (Pyodide) | Idle', and 'Ln 1, Col 1 lab_jupyter_launch_site_location.ipynb 3'.

CONTD...



PREDICTION OF LOGISTIC REGRESSION

TASK 4

Create a logistic regression object then create a GridSearchCV object `logreg_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
In [13]: parameters = {'C':[0.01,0.1,1],
                    'penalty':['l2'],
                    'solver':['lbfgs']}
```

```
In [14]: parameters = {'C':[0.01,0.1,1], 'penalty':['l2'], 'solver':['lbfgs']}# l1 lasso l2 ridge
lr=LogisticRegression()
```

We output the `GridSearchCV` object for logistic regression. We display the best parameters using the data attribute `best_params_` and the accuracy on the validation data using the data attribute `best_score_`.

```
In [16]: # Define the logistic regression model
logreg = LogisticRegression(max_iter=1000) # Increase max_iter to ensure convergence

# Define hyperparameter grid
param_grid = {
    'C': [0.01, 0.1, 1, 10, 100], # regularization strength
    'solver': ['liblinear', 'lbfgs'] # solvers compatible with LogisticRegression
}

# Create GridSearchCV object
grid_search = GridSearchCV(estimator=logreg, param_grid=param_grid, cv=5)

# Fit GridSearchCV on training data
grid_search.fit(X_train, Y_train)

# Output the GridSearchCV object
print(grid_search)

# Display the best hyperparameters
print("Best Parameters:", grid_search.best_params_)

# Display the best validation score (mean accuracy across folds)
print("Best Validation Accuracy:", grid_search.best_score_)
```

```
GridSearchCV(cv=5, estimator=LogisticRegression(max_iter=1000),
            param_grid={'C': [0.01, 0.1, 1, 10, 100],
                        'solver': ['liblinear', 'lbfgs']})
Best Parameters: {'C': 0.1, 'solver': 'lbfgs'}
Best Validation Accuracy: 0.8342857142857143
```

TASK 5

Calculate the accuracy on the test data using the method `score`:



PREDICTION OF LOGISTIC REGRESSION CONTD...

TASK 5

Calculate the accuracy on the test data using the method `score`:

```
In [17]: # Get the best estimator from GridSearchCV
best_logreg = grid_search.best_estimator_

# Calculate accuracy on the test set
test_accuracy = best_logreg.score(X_test, Y_test)

print("Test Set Accuracy:", test_accuracy)
```

Test Set Accuracy: 0.8333333333333334

Lets look at the confusion matrix:

```
In [19]: from sklearn.metrics import confusion_matrix

# Get predictions on the test set
Y_pred = best_logreg.predict(X_test)

# Compute the confusion matrix
cm = confusion_matrix(Y_test, Y_pred)

print("Confusion Matrix:")
print(cm)
```

Confusion Matrix:
[[3 3]
[0 12]]

Examining the confusion matrix, we see that logistic regression can distinguish between the different classes. We see that the problem is false positives.

Overview:

True Postive - 12 (True label is landed, Predicted label is also landed)

False Postive - 3 (True label is not landed, Predicted label is landed)



PREDICTION OF GRIDSEARCHCV OBJECT SVM

TASK 5

Calculate the accuracy on the test data using the method `score`:

```
In [17]: # Get the best estimator from GridSearchCV
best_logreg = grid_search.best_estimator_

# Calculate accuracy on the test set
test_accuracy = best_logreg.score(X_test, Y_test)

print("Test Set Accuracy:", test_accuracy)
```

Test Set Accuracy: 0.8333333333333334

Lets look at the confusion matrix:

```
In [19]: from sklearn.metrics import confusion_matrix

# Get predictions on the test set
Y_pred = best_logreg.predict(X_test)

# Compute the confusion matrix
cm = confusion_matrix(Y_test, Y_pred)

print("Confusion Matrix:")
print(cm)
```

Confusion Matrix:

```
[[ 3  3]
 [ 0 12]]
```

Examining the confusion matrix, we see that logistic regression can distinguish between the different classes. We see that the problem is false positives.

Overview:

True Positive - 12 (True label is landed, Predicted label is also landed)

False Positive - 3 (True label is not landed, Predicted label is landed)



PREDICTION OF SVM CONTD...

TASK 7

Calculate the accuracy on the test data using the method `score`:

```
In [23]: # Get the best SVM estimator from GridSearchCV
best_svm = svm_cv.best_estimator_

# Calculate accuracy on the test set
test_accuracy = best_svm.score(X_test, Y_test)

print("SVM Test Set Accuracy:", test_accuracy)
```

SVM Test Set Accuracy: 0.7777777777777778

We can plot the confusion matrix

```
In [24]: from sklearn.metrics import confusion_matrix

# Predict labels for the test set using the best SVM model
Y_pred_svm = best_svm.predict(X_test)

# Compute the confusion matrix
cm_svm = confusion_matrix(Y_test, Y_pred_svm)

print("SVM Confusion Matrix:")
print(cm_svm)
```

SVM Confusion Matrix:
[[3 3]
 [1 11]]



PREDICTION OF DECISION TREE

SVM Confusion Matrix:
[[3 3]
[1 11]]

TASK 8

Create a decision tree classifier object then create a `GridSearchCV` object `tree_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
In [26]: from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# Create a Decision Tree classifier object
tree = DecisionTreeClassifier(random_state=2)

# Define the hyperparameter grid
parameters = {
    'criterion': ['gini', 'entropy'], # function to measure quality of split
    'max_depth': [None, 2, 4, 6, 8, 10], # maximum depth of the tree
    'min_samples_split': [2, 5, 10], # minimum samples required to split a node
    'min_samples_leaf': [1, 2, 4] # minimum samples required at a leaf node
}

# Create GridsearchCV object with 10-fold cross-validation
tree_cv = GridSearchCV(estimator=tree, param_grid=parameters, cv=10)

# Fit the GridSearchCV object to the training data
tree_cv.fit(X_train, Y_train)

# Display the best parameters
print("Best Parameters:", tree_cv.best_params_)

# Display the best cross-validation score
print("Best CV Accuracy:", tree_cv.best_score_)

Best Parameters: {'criterion': 'gini', 'max_depth': None, 'min_samples_leaf': 4, 'min_samples_split': 2}
Best CV Accuracy: 0.8714285714285713
```

TASK 9

Calculate the accuracy of `tree_cv` on the test data using the method `score`:

```
In [27]: # Get the best estimator from GridSearchCV
best_tree = tree_cv.best_estimator_

# Calculate accuracy on the test set
test_accuracy = best_tree.score(X_test, Y_test)

print("Decision Tree Test Set Accuracy:", test_accuracy)
```



PREDICTION OF DECISION TREE CONTD...

TASK 9

Calculate the accuracy of `tree_cv` on the test data using the method `score`:

```
In [27]: # Get the best estimator from GridSearchCV
best_tree = tree_cv.best_estimator_

# Calculate accuracy on the test set
test_accuracy = best_tree.score(X_test, Y_test)

print("Decision Tree Test Set Accuracy:", test_accuracy)
```

Decision Tree Test Set Accuracy: 0.7777777777777778

We can plot the confusion matrix

```
In [28]: from sklearn.metrics import confusion_matrix

# Predict labels on the test set using the best Decision Tree
Y_pred_tree = best_tree.predict(X_test)

# Compute the confusion matrix
cm_tree = confusion_matrix(Y_test, Y_pred_tree)

print("Decision Tree Confusion Matrix:")
print(cm_tree)
```

Decision Tree Confusion Matrix:
[[3 3]
 [1 11]]



PREDICTION OF K NEAREST NEIGHBORS CONTD...

TASK 10

Create a `k nearest neighbors` object then create a `GridSearchCV` object `knn_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
In [29]: from sklearn.neighbors import KNeighborsClassifier
         from sklearn.model_selection import GridSearchCV

         # Create a KNN classifier object
         knn = KNeighborsClassifier()

         # Define the hyperparameter grid
         parameters = {
             'n_neighbors': [3, 5, 7, 9, 11], # number of neighbors
             'weights': ['uniform', 'distance'], # uniform or distance-based weighting
             'metric': ['euclidean', 'manhattan'] # distance metric
         }

         # Create GridSearchCV object with 10-fold cross-validation
         knn_cv = GridSearchCV(estimator=knn, param_grid=parameters, cv=10)

         # Fit the GridSearchCV object to the training data
         knn_cv.fit(X_train, Y_train)

         # Display the best parameters
         print("Best Parameters:", knn_cv.best_params_)

         # Display the best cross-validation score
         print("Best CV Accuracy:", knn_cv.best_score_)
```

```
Best Parameters: {'metric': 'manhattan', 'n_neighbors': 11, 'weights': 'uniform'}
Best CV Accuracy: 0.8482142857142858
```



PREDICTION OF K NEAREST NEIGHBORS CONTD...

TASK 11

Calculate the accuracy of knn_cv on the test data using the method `score`:

```
[30]: # Get the best estimator from GridSearchCV
      best_knn = knn_cv.best_estimator_

      # Calculate accuracy on the test set
      test_accuracy = best_knn.score(X_test, Y_test)

      print("KNN Test Set Accuracy:", test_accuracy)
```

KNN Test Set Accuracy: 0.8333333333333334

We can plot the confusion matrix

```
[31]: from sklearn.metrics import confusion_matrix

      # Predict labels on the test set using the best KNN model
      Y_pred_knn = best_knn.predict(X_test)

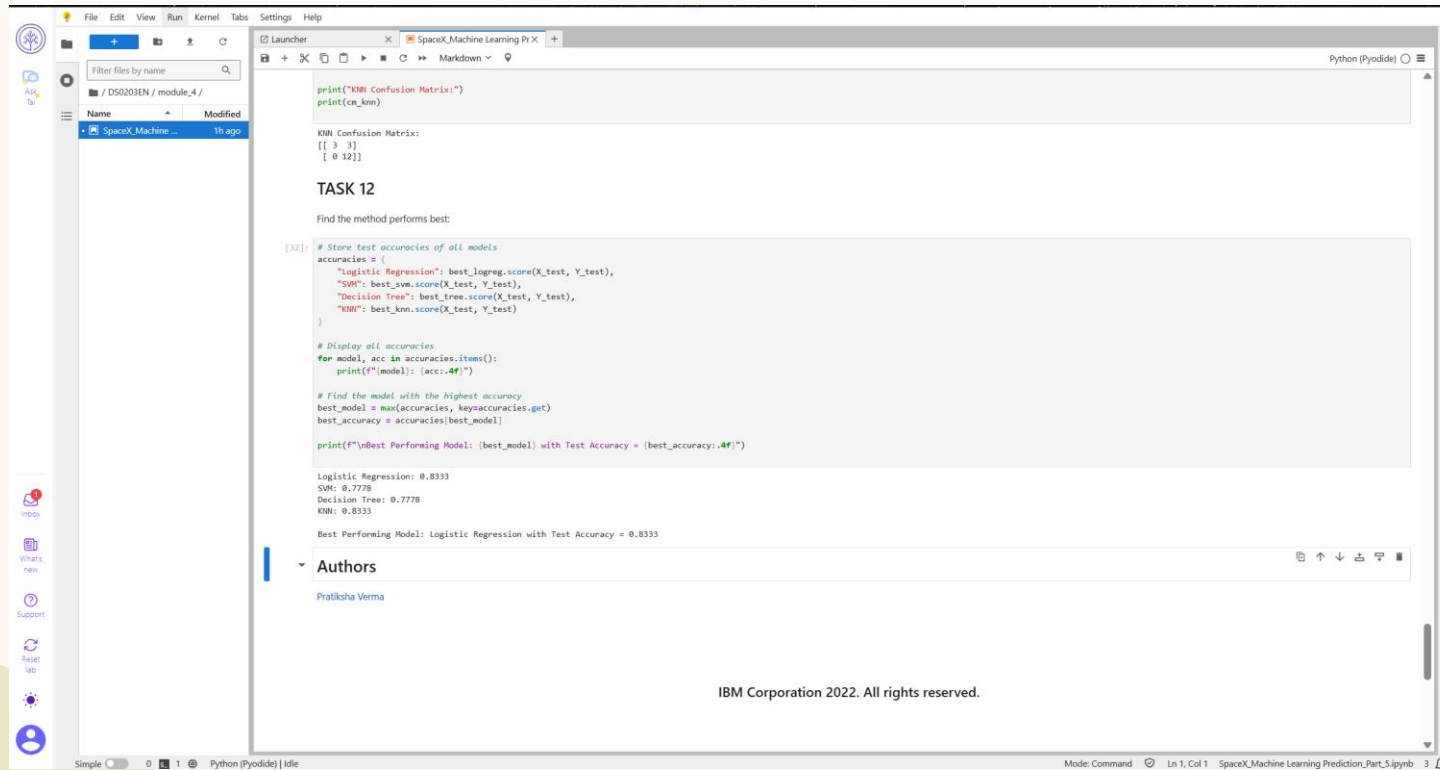
      # Compute the confusion matrix
      cm_knn = confusion_matrix(Y_test, Y_pred_knn)

      print("KNN Confusion Matrix:")
      print(cm_knn)
```

KNN Confusion Matrix:
[[3 3]
 [0 12]]



BEST METHOD (LOGISTIC REGRESSION)



```
File Edit View Run Kernel Tabs Settings Help
Launcher
SpaceX_Machine Learning PyX
Filter files by name
Name Modified
SpaceX_Machine... 1h ago

print("KNN Confusion Matrix:")
print(cm_knn)

KNN Confusion Matrix:
[[ 0  0]
 [ 0 12]]

TASK 12
Find the method performs best:

[32]: # Store test accuracies of all models
accuracies = {
    "Logistic Regression": best_logreg.score(X_test, Y_test),
    "SVM": best_svm.score(X_test, Y_test),
    "Decision Tree": best_tree.score(X_test, Y_test),
    "KNN": best_knn.score(X_test, Y_test)
}

# Display all accuracies
for model, acc in accuracies.items():
    print(f"{model}: {acc:.4f}")

# Find the model with the highest accuracy
best_model = max(accuracies, key=accuracies.get)
best_accuracy = accuracies[best_model]

print(f"\nBest Performing Model: {best_model} with Test Accuracy = {best_accuracy:.4f}")

Logistic Regression: 0.8333
SVM: 0.7778
Decision Tree: 0.7778
KNN: 0.8333

Best Performing Model: Logistic Regression with Test Accuracy = 0.8333

Authors
Pratiksha Verma

IBM Corporation 2022. All rights reserved.
```



WHY LOGISTIC REGRESSION IS BETTER?

Linear Separation of Classes:

Logistic Regression performs well when the classes in your dataset are linearly separable or nearly so. If your features clearly differentiate the classes, Logistic Regression can efficiently capture that pattern.

Less Overfitting:

Compared to Decision Trees or KNN, Logistic Regression is less prone to overfitting, especially on smaller or moderately sized datasets. This leads to better generalization on test data, hence higher accuracy.



WHY LOGISTIC REGRESSION IS BETTER?

Feature Scaling Compatibility:

Logistic Regression works well when features are standardized, as you did with your X dataset. Models like KNN are sensitive to feature scale, and unscaled features may reduce accuracy.

Simple and Robust Model:

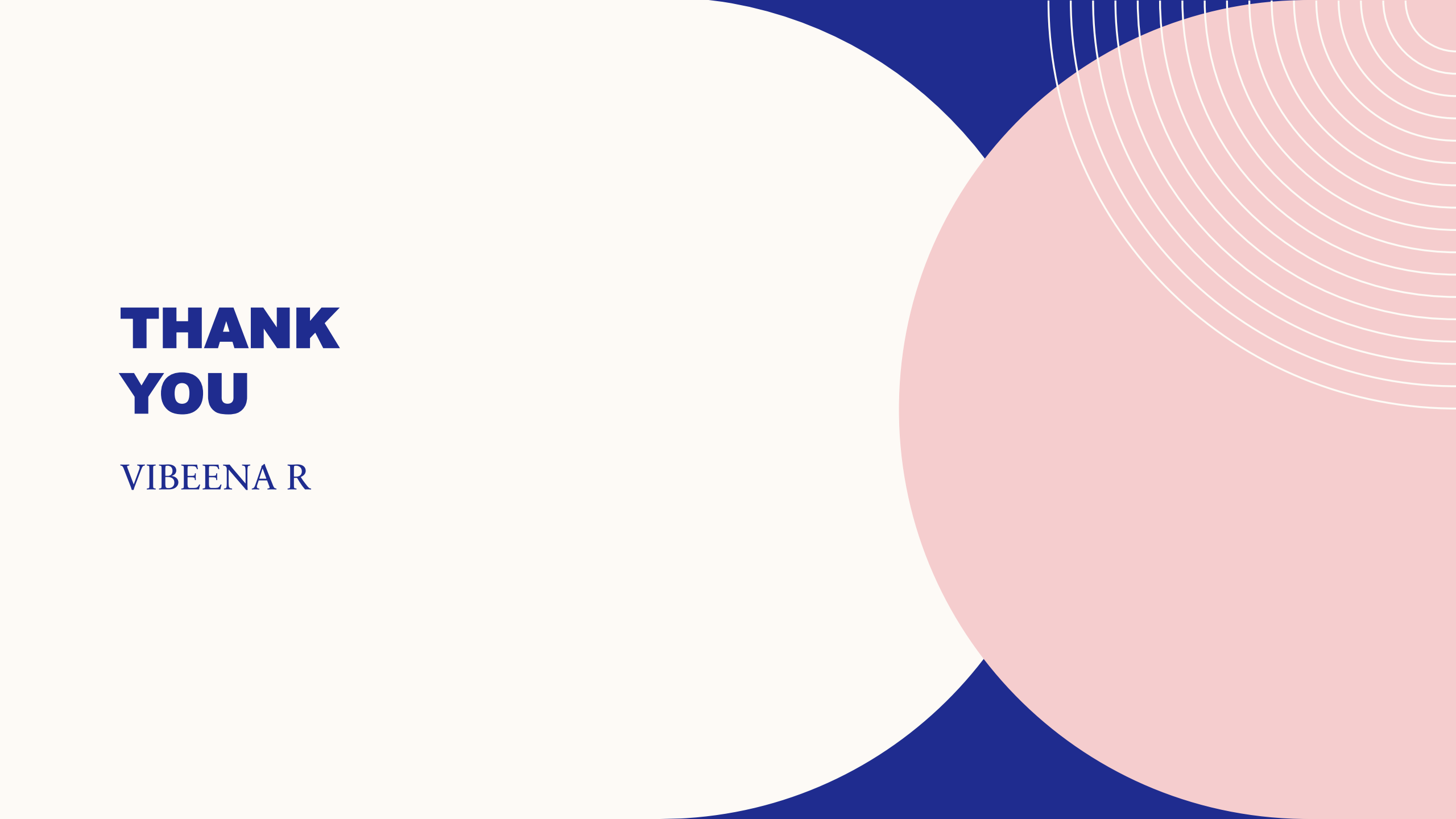
Logistic Regression is computationally efficient and robust for binary or multi-class classification, making it reliable for datasets without highly complex relationships.



CONCLUSION

This project demonstrates an **end-to-end workflow** from data collection and SQL-based exploration to EDA, visualization, and predictive modeling. By applying **machine learning models** such as Logistic Regression, SVM, Decision Tree, and KNN, we evaluated performance using accuracy and confusion matrices. The methodology highlights the importance of **data preparation, feature selection, and model tuning** in achieving reliable predictions.





**THANK
YOU**

VIBEENA R